

The computational tasks behind the layered-memory codes

what is computed, by which algorithm, at what cost, and where it could be much faster

July 2026 — working notes

Every experiment in this project is assembled from eight computational tasks. This note describes each task in a fixed format: what the task is, the formula it evaluates, its inputs and outputs, the algorithm we currently use, the cost of that algorithm and how the cost grows, and ideas for computing the same thing faster. Ideas that give identical results are marked [exact]; ideas that introduce a controlled error are marked [approx]. Section 1 defines all concepts and symbols; nothing is used before it is defined there.

1 Definitions

The data. The data is a sequence $x_1, x_2, \dots, x_n$ of *symbols*. In our experiments a symbol is an English word, and the sequence is the corpus `text8` with $n = 1.7 \cdot 10^7$ words. There are d possible symbols (the *alphabet size*); depending on the experiment, d ranges from 256 to the full number of distinct words, 253,855.

States. A predictor with memory works as follows. A fixed rule σ assigns to every position t a *state* $s_t = \sigma(x_1, \dots, x_{t-1})$, computed from the symbols before position t . Examples of rules used in this project: the state is the symbol immediately before position t ; or the pair of the two preceding symbols; or the symbol that occurred δ positions earlier (we call δ the *lag*). The predictor keeps one table of statistics per state, and at position t it predicts x_t using the table that belongs to the state s_t .

Count rows. Fix a state s . Among the positions t whose state equals s , count, for each symbol x , how many of these positions actually carried the symbol x :

$$m_x(s) = \#\{t : s_t = s \text{ and } x_t = x\}.$$

The vector $(m_1(s), \dots, m_d(s))$ is called the *row* of the state s . In words: the row of s records how often each symbol occurred at the positions where the state was s . We write s for the number of nonzero entries of a row when no confusion with the state is possible, $N = \sum_x m_x$ for the sum of the row's entries, and we call a symbol *observed in the row* if its entry is nonzero.

Profiles. The *profile* of a row is the list of its nonzero entries, sorted in increasing order, with the identities of the symbols discarded. For example, the row $(0, 3, 0, 1, 1)$ has the profile $(1, 1, 3)$. We write a profile as $\mathbf{m} = (m_1, \dots, m_k)$ with $0 < m_1 \leq \dots \leq m_k$. Two further quantities of a profile matter for costs: $\tilde{k}$, the number of *distinct* values among its entries (the example has $\tilde{k} = 2$: the values 1 and 3), and the multiplicity μ_c of each distinct value c (here $\mu_1 = 2, \mu_3 = 1$).

The estimator. All experiments use one estimator of probabilities from counts, the *layered mixture* of the companion paper. It is a Bayesian mixture: a prior distribution over probability vectors, updated by the observed counts. The prior is built in *levels*. At level L , attach to each symbol x a random weight

$$W_x = E_{x,1} \cdot E_{x,2} \cdots E_{x,L},$$

where all $E_{x,\ell}$ are independent random variables with the standard exponential distribution (density e^{-v} for $v > 0$). The random probability vector of level L is $p_x = W_x / (W_1 + \cdots + W_d)$. The larger L is, the more uneven these probability vectors tend to be, so L acts as a roughness parameter. The full estimator does not choose L ; it averages uniformly over $L = 1, \dots, L_{\max}$, with $L_{\max} = \lceil 2c^* \ln d \rceil$ and $c^* \approx 2.365$ a constant from the companion paper. In our runs $L_{\max}$ is between 33 and 59.

The central quantity. For a profile $\mathbf{m}$ and a level L , define

$$q_L(\mathbf{m}) = \mathbb{E} \left[p_{y_1}^{m_1} \cdot p_{y_2}^{m_2} \cdots p_{y_k}^{m_k} \right], \quad q_{\text{avg}}(\mathbf{m}) = \frac{1}{L_{\max}} \sum_{L=1}^{L_{\max}} q_L(\mathbf{m}), \quad (1)$$

where $y_1, \dots, y_k$ are any k distinct symbols and the expectation is over the random vector p of level L . Because the prior treats all symbols in the same way (it is exchangeable), the value does not depend on which distinct symbols are chosen — only the profile matters. This is the number the whole project runs on: $q_{\text{avg}}(\mathbf{m})$ is the probability that the estimator assigns to any one particular sequence of symbols whose counts form the profile $\mathbf{m}$. The compressed length contributed by one row is $-\log_2 q_{\text{avg}}$ of its profile; the length of a whole model is the sum of this over its rows. Because rows with equal profiles contribute equal amounts, all computations deduplicate rows by profile first; this typically shrinks the workload by a factor of 50 to 100.

The integral form. Eq. (1) is an expectation over d random weights; to compute it we turn it into a one-dimensional integral. The denominator $(W_1 + \cdots + W_d)^N$ is removed with the identity $A^{-N} = \frac{1}{\Gamma(N)} \int_0^\infty t^{N-1} e^{-tA} dt$ (valid for $A > 0$; here t is a real integration variable, a scalar). After this substitution the expectation splits into a product of one factor per symbol, because the weights of different symbols are independent. The result is exact:

$$q_L(\mathbf{m}) = \frac{1}{\Gamma(N)} \int_0^\infty t^{N-1} \varphi_0^{(L)}(t)^{d-k} \prod_{i=1}^k \varphi_{m_i}^{(L)}(t) dt, \quad (2)$$

where the function $\varphi_r^{(L)}$, which we call the *moment function*, is defined by

$$\varphi_r^{(L)}(t) = \mathbb{E} [Y^r e^{-tY}], \quad Y = E_1 E_2 \cdots E_L \quad (3)$$

with $E_1, \dots, E_L$ independent standard exponentials. In words: Y is a product of L independent exponential random variables, and $\varphi_r^{(L)}(t)$ is the expected value of $Y^r e^{-tY}$ — a function of the single real argument $t \geq 0$, one such function for each pair (L, r) . The factor $\varphi_0^{(L)}(t)^{d-k}$ in Eq. (2) collects the $d - k$ symbols that are not observed in the row (their count is $r = 0$).

The computational axis. All numerics happen on the logarithmic axis $u = \ln t$ (u is a scalar), restricted to a finite grid $u_1 < u_2 < \dots < u_G$; in the largest runs $G = 3.6 \cdot 10^4$. Taking logarithms of the integrand of Eq. (2) and substituting $t = e^u$ gives the *log-integrand*

$$\psi_L(u) = Nu + (d - k) \ln \varphi_0^{(L)}(e^u) + \sum_{\text{distinct values } c \text{ of } \mathbf{m}} \mu_c \ln \varphi_c^{(L)}(e^u), \quad (4)$$

so that $q_L(\mathbf{m}) = \frac{1}{\Gamma(N)} \int_{-\infty}^{\infty} e^{\psi_L(u)} du$. Two exact facts anchor the numerics. First, at level one the moment function has the closed form $\varphi_r^{(1)}(t) = \Gamma(r + 1)/(1 + t)^{r+1}$, and the whole integral at $L = 1$ reduces to a classical closed form (the Dirichlet–multinomial); no numerics are needed there. Second, as $t \rightarrow 0$ the moment function tends to the constant $\Gamma(r + 1)^L$; therefore, to the left of the grid, $\psi_L(u) = Nu + \text{const} + O(e^u)$, and the left tail of the integral is available in closed form.

Symbols used in the cost formulas, with the values measured in our runs:

symbol	meaning	measured
n	length of the corpus, in symbols	$1.7 \cdot 10^7$
d	alphabet size	$256 - 2.6 \cdot 10^5$
$L_{\max}$	number of levels, $\lceil 2c^* \ln d \rceil$	$33 - 59$
G	number of grid points on the u axis	$3.6 \cdot 10^4$
Q	quadrature order used to build tables (T1)	96
$\mathcal{R}$	set of count values for which tables are needed	$10^3 - 5 \cdot 10^3$ values
$\mathcal{N}$	number of states of a model	$10^3 - 2.5 \cdot 10^5$
P	number of nonzero row entries, summed over rows	up to $1.5 \cdot 10^7$
U	number of distinct profiles after deduplication	$2.6 \cdot 10^4 - 7 \cdot 10^4$
N	sum of one row's entries	$1 - 10^6$
k	number of nonzero entries of one row	$1 - 4,096$
$\tilde{k}$	number of distinct values in one row	$1 - \sim 300$
C	number of table refreshes of a sequential code (T8)	$8 - 32$
D	number of lag predictors combined (T6, T7)	6
K	number of weight vectors in a combination grid	$\sim 10^2$

2 T1 — Building the tables of moment functions

Task. Compute the moment functions $\varphi_r^{(L)}$ of Eq. (3) numerically, for every level $L \leq L_{\max}$ and every count value r that occurs in the data, and store their logarithms on the u grid. This is done once per alphabet size; the stored tables are then read by tasks T2 and T3.

Formula. Write $Y = Y' \cdot E_L$, where Y' is the product of the first $L - 1$ exponentials. Taking the expectation over E_L first gives a recursion that connects level ℓ to level $\ell - 1$:

$$\varphi_r^{(\ell)}(t) = \int_0^\infty e^{-v} v^r \varphi_r^{(\ell-1)}(tv) dv, \quad \varphi_r^{(1)}(t) = \frac{\Gamma(r + 1)}{(1 + t)^{r+1}}. \quad (5)$$

Input. The set $\mathcal{R}$ of count values that occur anywhere in the data; the number of levels $L_{\max}$; the grid $u_1, \dots, u_G$.

Output. The numbers $\ln \varphi_r^{(L)}(e^{u_g})$ for all $r \in \mathcal{R}$, $L \leq L_{\max}$, $g \leq G$ — an array of $|\mathcal{R}| \times L_{\max} \times G$ values, stored on disk.

Algorithm. For each r separately: start from the closed form at level one; then, for $\ell = 2, 3, \dots, L_{\max}$, evaluate the integral in Eq. (5) at every grid point by Gauss–Laguerre quadrature with Q nodes. The quadrature needs the previous level at the shifted arguments tv_j ; these values are obtained by linear interpolation on the u grid, using the exact limit $\Gamma(r+1)^{\ell-1}$ to the left of the grid. The work for different values of r is independent, so it is spread over all processor cores; when the data later produces a count value that has no table yet, that table is built on demand and added to the store.

Cost and scaling. Building: about $|\mathcal{R}| \cdot L_{\max} \cdot G \cdot Q$ interpolate-and-multiply steps; at the largest settings this is $\approx 10^{12}$ steps, measured as 45 minutes on 64 cores. Storage: $8 \cdot |\mathcal{R}| \cdot L_{\max} \cdot G$ bytes, which reached 85 GB. Growth: on a larger corpus, $|\mathcal{R}|$ grows slowly (a new count value must appear for the first time, which becomes rare); G grows in proportion to the length of the u interval, which grows like the logarithm of the largest count; $L_{\max}$ grows like the logarithm of d . The recurring cost after building is *reading* the tables, and the storage layout determines whether reading is fast (a lesson learned on the shared cluster file system).

Ideas for speedup. (1) [exact] Keep one shared table store for all experiments; when we audited the disk we found 219 GB of identical tables stored by different runs. (2) [exact] Store the array level by level in single files, in single precision: four times smaller, and a task that needs level L opens one file instead of $|\mathcal{R}|$ files. (3) [approx] For large r the integrand of Eq. (5) is sharply peaked and $\varphi_r^{(L)}$ approaches a smooth limiting shape; beyond some threshold the table could be replaced by this limiting formula. The error must be worked out before this is used.

3 T2 — Evaluating the probability of one profile

Task. Given one profile, compute $\log q_L$ for every level L , and from these $\log q_{\text{avg}}$. This is the innermost computation of the project; we call it the *atom*. A static model invokes it once per distinct profile; a sequential model invokes it again at every table refresh (T8).

Formula. Eqs. (1)–(4): $\log q_L(\mathbf{m}) = \log \int e^{\psi_L(u)} du - \ln \Gamma(N)$.

Input. The profile, given as its distinct values c with multiplicities μ_c ; the tables of T1.

Output. The vector $(\log_2 q_L(\mathbf{m}))_{L=1..L_{\max}}$ and the average $\log_2 q_{\text{avg}}(\mathbf{m})$.

Algorithm. For each level: (i) assemble the log-integrand ψ_L on the grid by Eq. (4); this reads $\hat{k} + 1$ rows of the table and costs $O(G\hat{k})$ additions. (ii) Find all local maxima of ψ_L that lie within a fixed window below the global maximum. (The integrand can have two peaks: for deep levels and large counts a second peak appears far to the left.) (iii) Refine each maximum by solving $\psi'_L(u) = 0$ in a bracket around it, and add the peak contributions by Laplace’s method (a Gaussian approximation around each peak, summed in the log domain). (iv) Add the closed-form left tail from Section 1. Profiles with $N \leq 1$, and the level $L = 1$, are returned from closed forms

without any of this. Batches of profiles are processed in parallel: the profiles are split over worker processes, and each level’s table rows are handed to the workers through shared memory.

Cost and scaling. Step (i) dominates: $O(L_{\max} G \tilde{k})$ per profile — between $4 \cdot 10^6$ operations for a poor profile ($\tilde{k} = 2$) and $2 \cdot 10^8$ for a rich one ($\tilde{k} \approx 100$). Measured: $7 \cdot 10^4$ distinct profiles at $L_{\max} = 59$ in 17 minutes on 64 cores.

Ideas for speedup. (1) [exact] *Remember where the peaks are.* The full sweep over the grid exists only to locate the maxima reliably. The locations move very little when a profile changes slightly — in particular, between two refreshes of the same row. Storing the peak locations and starting the bracketed solve from the stored value would let the algorithm evaluate ψ_L only on a small window around each peak, replacing G by a window size of order 10^2 ; a full sweep remains as a fallback when the solve fails. Potential factor: up to 300 on step (i). (2) [approx] *Truncate the sum over levels.* The sum in Eq. (1) is dominated by a few levels around its largest term, and the terms decay quickly away from it. Evaluating the dominant levels exactly and bounding the rest would replace $L_{\max}$ by an effective count of 5 to 10. (3) [approx] Use coarser grids for profiles with small N , whose integrands are wide and smooth; choose the grid per profile from a target accuracy.

4 T3 — The prediction table of one row

Task. Turn one row into a probability distribution for the symbol at the next position. This is the workhorse of the sequential codes of T6–T8: at every refresh, every row of every predictor is converted into such a table.

Formula. Let $\mathbf{m}$ be the row’s profile and let c be the row’s entry for some symbol ($c = 0$ if the symbol is not observed in the row). Write $\mathbf{m} \oplus c$ for the profile obtained from $\mathbf{m}$ by increasing one entry with value c by one (for $c = 0$: by appending a new entry 1). The estimator’s probability that the next symbol is that particular symbol is

$$p = \frac{q_{\text{avg}}(\mathbf{m} \oplus c)}{q_{\text{avg}}(\mathbf{m})}. \quad (6)$$

The value depends on the symbol only through its count c , so the table has only $\tilde{k} + 1$ distinct entries (one per distinct count value, and one shared by all unobserved symbols). The entries sum to one exactly; in practice they are renormalized to absorb the numerical error of T2, measured at about $5 \cdot 10^{-5}$ relative.

Input. The row’s counts; the tables of T1.

Output. The logarithms of the $\tilde{k} + 1$ distinct probabilities, and the assignment of each symbol to its probability.

Algorithm. Today: one T2 evaluation for $\mathbf{m}$ and one for each of the $\tilde{k} + 1$ profiles $\mathbf{m} \oplus c$; results are cached by profile, so a profile that appears again (in another row, or at another refresh) is not recomputed.

Cost and scaling. $\tilde{k} + 2$ atom evaluations per row, i.e. $O(L_{\max} G \tilde{k}^2)$ — note the square. Measured: the pooled run at $d = 1,024$ with $C = 32$ refreshes needed $5.9 \cdot 10^6$ atom evaluations, which is the dominant cost of that run; the cache helps little because an active row’s profile is different at every refresh.

Ideas for speedup. (1) [exact; the largest single lever in this document] *Compute all the ratios from one integrand.* The profiles $\mathbf{m} \oplus c$ differ from $\mathbf{m}$ in a single factor of the integrand in Eq. (2). Concretely,

$$q_L(\mathbf{m} \oplus c) = \frac{1}{\Gamma(N+1)} \int e^{\psi_L(u)} \cdot e^u \frac{\varphi_{c+1}^{(L)}(e^u)}{\varphi_c^{(L)}(e^u)} du,$$

with ψ_L the log-integrand of the *base* profile, Eq. (4). So: assemble ψ_L once, at cost $O(G\tilde{k})$; then every one of the $\tilde{k} + 1$ entries is a single weighted sum over the grid, at cost $O(G)$ each. The cost per row falls from $O(G\tilde{k}^2)$ to $O(G\tilde{k})$ — a factor $\tilde{k}$, which is about 100 exactly for the rich rows that dominate — and the result is the same numbers. (2) [exact] *Update the integrand instead of rebuilding it.* Between two refreshes only a few entries of a row change. If the row’s ψ_L is kept in memory, it can be updated by adding $\ln \varphi_{c'}^{(L)} - \ln \varphi_c^{(L)}$ for each changed entry: cost proportional to the number of changes, not to $\tilde{k}$. Keeping ψ_L for one row costs $G \times L_{\max}$ single-precision numbers (≈ 8 MB at the largest settings), which is affordable for the busy rows — and the busy rows are exactly the expensive ones. (3) The ideas of T2 apply on top.

5 T4 — Counting

Task. Read the corpus once and produce, for a given state rule σ , all rows, their profiles, and the assignment of each row to its profile.

Formula. $m_x(s)$ as in Section 1, for every state s that occurs; then profiles and deduplication.

Input. The corpus as n integers; the rule σ .

Output. $\mathcal{N}$ rows with P nonzero entries in total; the list of U distinct profiles; the map from rows to profiles.

Algorithm. One pass over the corpus with a hash table from (state, symbol) pairs to counters; then one pass over rows to sort each row’s nonzero entries and to deduplicate profiles.

Cost and scaling. Time is linear in n and negligible (minutes). *Memory* is the problem: the hash table costs 30 to 60 bytes per nonzero entry, measured at about 200 GB for $P \approx 1.5 \cdot 10^7$ (all contexts of depth up to two at full alphabet). This is what currently forbids depth three. After deduplication the downstream tasks see only $U \ll P$ profiles; measured examples: $4.4 \cdot 10^6$ rows collapse to $7 \cdot 10^4$ profiles.

Ideas for speedup. (1) [exact] Count on disk: sort the (state, symbol) pairs externally and merge equal pairs. Standard technology; memory becomes constant, time $O(n \log n)$ with small constants. (2) [exact] Build a suffix array of the corpus (a sorted index of all its positions, storable in a few bytes per position). Every context of every depth corresponds to one contiguous interval

of the suffix array, so the counts for *all* depths at once can be read off from it. This is the natural route to depth three and beyond. (3) [exact] Replace hash tables of Python objects by fixed-width integer arrays; this alone reduces the memory constant by an order of magnitude.

6 T5 — Aggregation

Task. Combine the atoms into the reported numbers: the compressed length of a model, the mixture over a family of models, the posterior over the family.

Formula. The length of one model is $\sum_{\text{rows}} -\log_2 q_{\text{avg}}(\text{profile of the row})$. For the context-tree family the mixture over all prunings is computed by the recursion $\beta(v) = \frac{1}{2}q_{\text{avg}}(\mathbf{m}_v) + \frac{1}{2}\prod_w \beta(w)$, where v runs over observed contexts, $\mathbf{m}_v$ is the row of v , and w runs over the observed one-symbol extensions of v . The mixture over any finite family of K models is the log-sum-exp of their total log-probabilities minus $\log_2 K$, and the posterior over the family is the normalized exponential of the same totals.

Input/Output/Algorithm/Cost. Inputs are the per-profile atoms (T2) and the context structure; outputs are bits-per-symbol figures and posteriors. Single passes over rows and contexts; cost $O(N)$ or $O(P)$; measured in seconds. No scaling concern.

7 T6 — Combining predictors by weighted sums

Task. Given D lag predictors and the no-memory predictor, code the corpus under *every* candidate weight vector of a grid simultaneously, in one pass.

Formula. At position t , predictor e supplies the probability $p_{e,t}$ that it assigned to the symbol that actually occurred at t (read from its current prediction table, T3). For each weight vector $\lambda = (\lambda_0, \dots, \lambda_D)$ of a grid of size K (nonnegative entries summing to one):

$$\text{bits}(\lambda) = \sum_t -\log_2 \left(\sum_{e=0}^D \lambda_e p_{e,t} \right).$$

Input. The $n \times (D+1)$ array of the numbers $p_{e,t}$; the grid of weight vectors.

Output. K compressed lengths, one per weight vector; their mixture and posterior (T5).

Algorithm and cost. One matrix product of the p array with the weight matrix per block of positions, then logarithms: $O(n(D+1)K)$ multiplications, about 10^{10} at our settings — minutes. Linear in everything; never a bottleneck.

8 T7 — Combining predictors by tempered products

Task. The same as T6 for the multiplicative rule. The product of probabilities must be renormalized *over the whole alphabet at every position*, and this normalization is the expensive part.

Formula. Let q_0 be the no-memory predictor’s table and $p_j(x)$ the probability that lag predictor j currently assigns to symbol x . For an exponent vector $\beta = (\beta_1, \dots, \beta_D)$ from a grid of size K_{prod} :

$$p_\beta(x) = \frac{s_\beta(x)}{Z}, \quad s_\beta(x) = q_0(x) \prod_{j=1}^D \left(\frac{p_j(x)}{q_0(x)} \right)^{\beta_j}, \quad Z = \sum_{x=1}^d s_\beta(x),$$

where Z is recomputed at every position (the tables p_j depend on the current states).

Input. The prediction tables (T3) of the D lag predictors and of q_0 ; the exponent grid.

Output. K_{prod} compressed lengths.

Algorithm today, and cost. Dense: at every position, form the d numbers $\ln s_\beta(x)$ and sum their exponentials. Cost $O(ndDK_{\text{prod}})$: measured 68 minutes at $d = 4,096$; at $d = 10^6$ and $n = 1.5 \cdot 10^8$ it would be $\sim 10^{16}$ operations. **This is the only part of the project whose cost is proportional to the alphabet size at every position.**

Idea for speedup. [exact; removes the factor d] By Eq. (6), each prediction table consists of one shared value for all symbols not observed in the row, plus individual values for the observed ones. Let S be the union, over the D predictors, of the sets of symbols observed in their current rows; $|S|$ is a few hundred to a few thousand, far below d . For every symbol outside S , each factor of $s_\beta(x)$ takes its shared value, so $s_\beta(x) = (\text{const}) \cdot q_0(x)^\gamma$ with $\gamma = 1 - \sum_j \beta_j$. Therefore

$$Z = (\text{const}) \cdot \left(T_\gamma - \sum_{x \in S} q_0(x)^\gamma \right) + \sum_{x \in S} s_\beta(x), \quad T_\gamma = \sum_{x=1}^d q_0(x)^\gamma,$$

and the power sums T_γ are computed once per refresh. The normalization then costs $O(|S|D)$ per position instead of $O(dD)$ — exact, and it makes the multiplicative rule feasible at any alphabet size.

9 T8 — Scheduling the refreshes of a sequential code

Task. A sequential code reads the corpus once and predicts each position from the counts gathered so far. Recomputing every prediction table at every position would be exact but hopeless; instead the tables are *refreshed* at chosen moments and used unchanged in between. Any refresh schedule that depends only on the past yields a valid code; using slightly outdated tables only lengthens the output a little. The schedule determines how many times T3 runs, and is therefore the multiplier that separates static runs (minutes) from sequential runs (hours).

Current schedule and cost. A global clock: all rows are refreshed at C evenly spaced positions. T3 work: C times the number of active rows; measured $5.9 \cdot 10^6$ atom evaluations at $d = 1,024$, $C = 32$.

Ideas for speedup. (1) [valid code; staleness controlled] *Refresh on growth*. Rebuild a row’s table when the row’s total N has grown by a fixed factor $1 + \epsilon$ since the last rebuild. A row that ends with total N is then rebuilt only $\log_{1+\epsilon} N$ times — logarithmic instead of C — and the table a position actually uses is never based on counts more than a factor $1 + \epsilon$ behind, which is the kind of error that matters for prediction quality. The trigger depends only on the past, so the code remains valid. Combined with ideas T3(1) and T3(2), the estimated overall gain on sequential runs is two to three orders of magnitude. (2) [exact] Distribute rows over cluster nodes; rows are independent once the schedule is fixed.

Scaling summary

Consider a corpus ten times larger ($n \approx 1.4 \cdot 10^8$, alphabet $\sim 10^6$). With today’s algorithms: T4 would need terabytes of memory (it already blocks depth three); T3 under the schedule of T8 would take weeks; T7 in its dense form is out of the question; T1 and T2 grow only through $|\mathcal{R}|$, G and $L_{\max}$ and remain manageable. With the ideas marked [exact] — one shared integrand per row (T3), integrand updates (T3), remembered peaks (T2), the sparse normalization (T7), refresh on growth (T8), counting on disk or via a suffix array (T4), and distribution over nodes — each task returns to the scale of node-hours. The ideas marked [approx] (level truncation, limiting formulas for large counts, coarser grids) are held in reserve, each with an error that can be checked against the exact computation on samples.

Suggested order of discussion: T3(1), T7, T8(1), T2(1), T4(2); the approximations only if something is still too slow after these.